



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



WHAT IS **GARBAGE COLLECTION** IN JAVA?



- Memory Management Technique in Java.
- Technically, it is a memory deallocation process in Java.
- In Java, the JVM (Java Virtual Machine) is responsible for memory management.
- JVM allocates and deallocates memory based on our Java code.
- Java has a Garbage Collector that is part of the JVM.
- When the JVM runs, the garbage collector identifies unused objects and removes them from memory.
- It removes unused objects from the heap memory, which is called garbage collection.
- This entire process is referred to as garbage collection.

Example:

- Let's say you have a phone with 128 GB memory and lots of unwanted videos.
- You delete the unwanted videos to manage the phone's memory.
- Similarly, garbage collection is a process of cleaning up unwanted memory in a Java application.

NOTE:

- Using the new operator, we can allocate memory for a new object.
- We can manually trigger garbage collection using `System.gc()`.



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



CASE STUDY BASED QUESTIONS

Case Study Based Questions: Different Ways to Achieve Garbage Collection



#1: By Default

-  Garbage collection happens automatically.
-  The garbage collector is a part of the JVM.
-  By default, the garbage collector periodically checks the heap memory to:
 - Identify unused objects.
 - Remove them from memory.

#2: Using `System.gc()`

-  You can request the JVM to call the garbage collector manually.
-  Use the `System.gc()` method to perform garbage collection when needed.

#3: Have You Ever Used `System.gc()`?

- "I never used it directly."
- Instead, I rely on: finally block.
- try-with-resources for better resource management.

Case Example: (Based On Real time Experience)

- Once, I faced a memory issue where I had to increase the heap size.
- In server configuration, I updated the `-Xms<size>` property.
- This helped me fix an `OutOfMemoryError` issue.



Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



BASIC RULE FOR GARBAGE COLLECTION

Basic Rule for Garbage Collection 📄

How does the Garbage Collector pick objects?

- It picks, Nullified objects.
- Dereferenced objects.
- Objects that are no longer needed after some operation.

Example:

- If an object is no longer required, we can nullify it so it can be picked up by the garbage collector during the process.

Best Practice: ✅

- Yes, it is a good practice to nullify objects that are no longer needed in your Java code.
- This helps optimize memory management and improves the performance of your application.

Have You Ever Faced Out Of Memory in Your Application? 🛠️💻

- ✅ Yes, I have faced it.
- 🧠 The memory allocated for the heap was insufficient.

🔧 Solution:

- I updated the heap configuration to resolve the issue.
- Increased the maximum heap size in the server configuration.

📄 Details:

Used the JVM properties:

- -Xms<initial size> (initial heap size).
- -Xmx<max size> (maximum heap size).
- Updated the -Xmx property to allocate a larger max size for the heap.



⚠️ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: @javatechcommunity.com - support@javatechcommunity.com

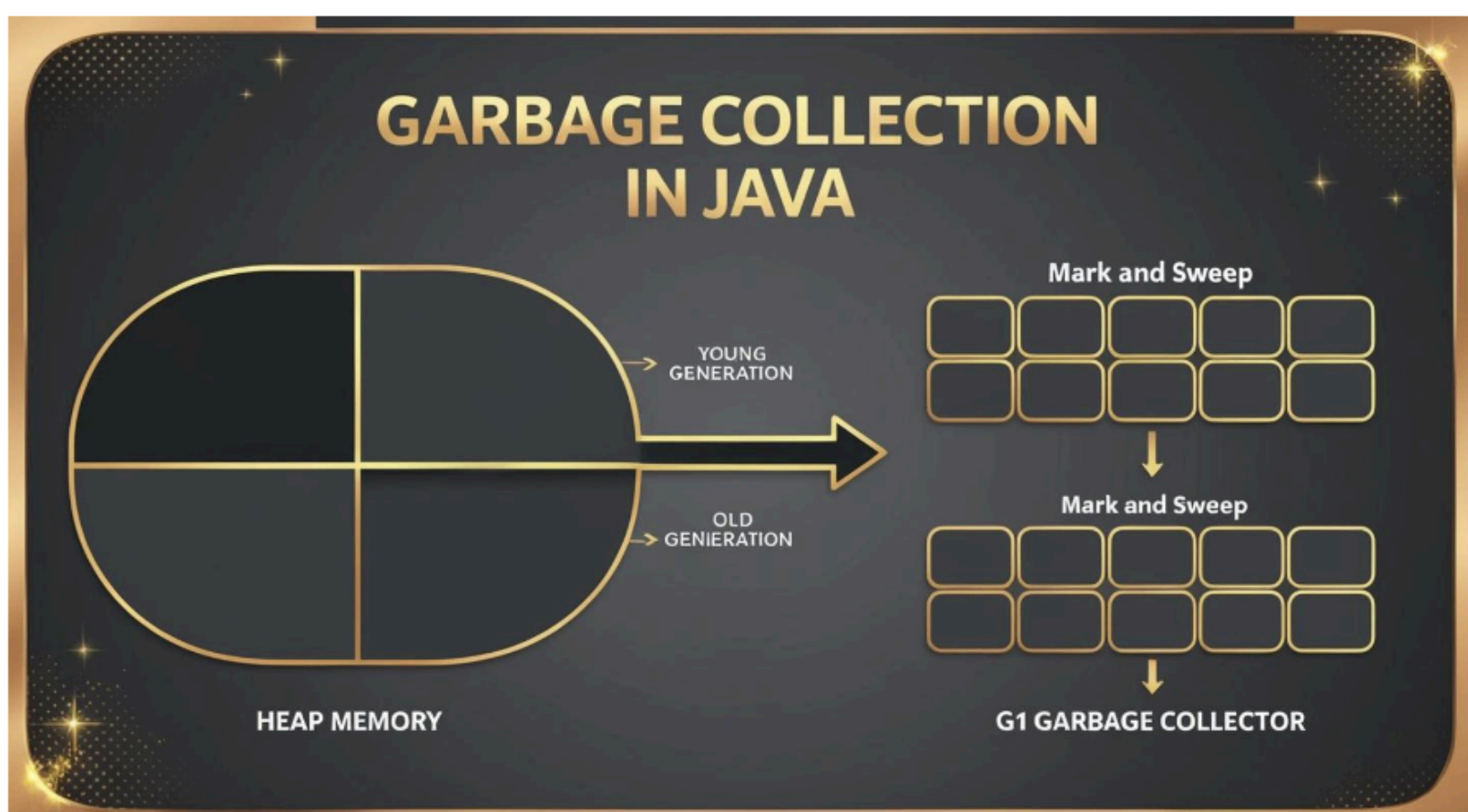


Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



GARBAGE COLLECTION ALGORITHMS IN JAVA 🗑️♻️

Garbage Collection algorithms in Java 🗑️♻️



Mark and Sweep Algorithm 🗑️🖌️

- ️ ✅ **Mark Phase:** JVM marks all active objects in the heap.
- ️ 🗑️ **Sweep Phase:** JVM frees up memory occupied by unmarked (inactive) objects.
- ️ ⚠️ **Limitation:** Not suitable for huge heaps.

G1 Garbage Collector 🚀🔧

- ️ Designed specifically for large heaps.
- ️ 🚀 Improves execution time for large memory spaces.
- ️ 🔄 Divides the heap into smaller regions to optimize memory management.
- ️ ⚙️ Uses concurrent technology to deallocate memory in parallel.



⚠️ **Copyright Notice:**

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com



Join us at javatechcommunity.com for our Premium Java Interview Kit and get referrals for Java-based roles – absolutely FREE!



PURPOSE OF THIS DOCUMENT

- **Use this PDF as a quick last-minute revision guide only.**
- **For more details, please perform R&D and gain a deeper understanding of how memory architecture works.**
- **Once the interview series is complete, we will cover this topic in detail.**



⚠ Copyright Notice:

This document is strictly for personal use only and not for redistribution or commercial purposes.

If you wish to use any part of this document, you must credit the source and creator: [@javatechcommunity.com](https://javatechcommunity.com) - support@javatechcommunity.com